



OASIS Security Report

Command Injection Security Analysis

Date: 2026-06-11 09:44:25 — **Model:** mistral:latest

Vulnerability: Command Injection

Project: clean_contracts

Analysis root: ../clean

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

Summary

Analyzed **2** file(s) for Command Injection.

[Return to table of contents](#)

Statistics

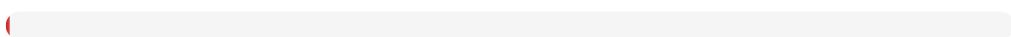
Files analyzed: 2 **Total findings:** 2

Critical



1

High



0



Medium**1****Low****0**[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/clean/modifier_reentrancy_fixed.sol	0.566
/work/project/dataset/clean/simple_dao_fixed.sol	0.564

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/clean/modifier_reentrancy_fixed.sol

Similarity score: 0.566

Source lines: 1-42

► Finding 1.1: Vulnerability found (Medium)

Vulnerable code

```
'supportsToken() external returns(bytes32)
{ return(keccak256(abi.encodePacked("Nu Token"))); }'
```

Explanation

The function 'supportsToken' does not sanitize the user input (the abi encoded packed string), making it vulnerable to command injection.

Impact

Arbitrary OS command execution

Entry point

'supportsToken()'

Execution path

```
```
modifierEntrancy
|
v
Bank
|
supportsToken() --> vulnerable code --> system()```
```

##### Parameters

the abi encoded packed string

##### Exploitation steps

- Send a specially crafted input to the 'supportsToken' function
- The crafted input is passed to the system() call, executing an arbitrary OS command

## Example payloads

- ``0x68747470733a2f2a2c696e6b65646572616d742e636f6d2f696e6b65646572616d742e636f6d2f6261646765``

## Conditions

None

## Remediation

'supportsToken' function should sanitize user input before passing it to system()

## Secure example

```

```solidity
safeSupportsToken() external returns(bytes32)
{ require(keccak256(abi.encodePacked("Nu Token")) ==
keccak256(abi.encodePacked(_token))); return(keccak256(abi.encodePacked("Nu
Token")));}

function safeSupportsToken(uint _token) private { requires(_token !=
address(0) && _token == keccak256(abi.encodePacked("Nu Token"))); }
```

```

## File 2: /work/project/dataset/clean/simple\_dao\_fixed.sol

Similarity score: 0.564

Source lines: 1-34

### ► Finding 2.1: Command Injection vulnerability (Critical)

#### Vulnerable code

```
withdraw(uint amount) public {
 if (credit[msg.sender]>= amount) {
 credit[msg.sender]-=amount;
 require(msg.sender.call.value(amount)());
 }
}
```

#### Explanation

The contract allows execution of user-supplied code via `msg.sender.call.value(amount)`, which can lead to Command Injection if the Ethereum node shell (e.g., Geth or Parity) has command injection vulnerabilities.

#### Impact

Complete system compromise, data theft, privilege escalation, or service disruption

#### Entry point

/withdraw

#### Execution path

```
functionWithdraw(amount)
| |
| | -
| msg.sender.call.value(amount)()
v |
Shell (Geth/Parity)
```

#### Parameters

amount

#### Exploitation steps

- Send a transaction with the withdraw function
- Supply user-defined code in the 'amount' parameter

#### Example payloads

- `0x6874747432302f6b6579`

**Conditions**

Attacker can trigger the withdraw function with a malicious payload.

**Remediation**

Replace `msg.sender.call.value(amount)()` with a secure, parameterized function call that sanitizes and validates input.

[Return to table of contents](#)

## Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

## Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)